

Report conversation

given a github lib of the form owner/repo, use the pygithub lib to get version number

To get the version number of a GitHub repository using the `PyGithub` library in Python, you typically need to check the tags or releases of the repository, as that's where version numbers are often stored. Here's a basic example of how you might do this:

1. First, install `PyGithub` if you haven't already. You can do this using `pip`:

```
bash
```

[Copy code](#)

```
pip install PyGithub
```

2. Then, use the following script as a starting point. Note that you'll need a GitHub access token to use the API:

```
python
```

[Copy code](#)

```
from github import Github
```

```
# Replace with your GitHub access token
```

```
access_token = "YOUR_ACCESS_TOKEN"
```

```
# Replace with the repository you're interested in, e.g., "owner/repo"
```

```
repo_name = "owner/repo"
```

```
# Create a Github instance:
```

```
g = Github(access_token)
```

```
# Get the repository
```

```
repo = g.get_repo(repo_name)
```

```
# List all tags
```

```
tags = repo.get_tags()
```

```
# Assuming the most recent tag is the latest version
```

```
latest_version = tags[0].name if tags.totalCount > 0 else "No version t
```

```
print(f"Latest version: {latest_version}")
```

3. In this script, replace "YOUR_ACCESS_TOKEN" with your personal access token and "owner/repo" with the GitHub repository's owner and name.
4. Run the script to get the latest version number.

Remember, this script assumes the latest tag corresponds to the latest version, which is a common practice but might not always be the case for every repository. You might need to adjust the logic depending on the specific repository's versioning system.

similarly, given a python pkg name, look up pypi and get list of last 10 version numbers

To fetch the last 10 version numbers of a Python package from PyPI (the Python Package Index), you can use the `requests` library to access the PyPI JSON API. Here's a step-by-step guide on how to do it:

1. Install the `requests` library if you haven't already. You can install it using pip:

```
bash
```

 Copy code

```
pip install requests
```

2. Use the following Python script as a guide. This script fetches the package data from PyPI and extracts the last 10 version numbers:

```
python
```

 Copy code

```
import requests
```

```
# Replace with the name of the package you're interested in
package_name = "package_name"
```

```
# PyPI URL for the package's JSON data
url = f"https://pypi.org/pypi/{package_name}/json"
```

```
try:
```

```
# Send a request to the URL
response = requests.get(url)
response.raise_for_status() # Will raise an exception for HTTP error

# Extract the data in JSON format
data = response.json()

# Extract version numbers from the data
versions = sorted(data["releases"].keys(), key=lambda s: list(map(int, s.split('.'))))

# Get the last 10 versions
last_10_versions = versions[:10]

print(f"Last 10 versions of {package_name}: {last_10_versions}")
except requests.RequestException as e:
    print(f"Error fetching data from PyPI: {e}")
```

3. Replace "package_name" with the name of the Python package you're interested in.
4. Run the script to get the last 10 version numbers of the specified package.

This script assumes that the version numbers are in a format that can be sorted in a meaningful way. If the package uses a non-standard version numbering system, you may need to adjust the sorting logic. The `requests` library is used to fetch the data, and the script handles basic error checking for network-related issues.